WEB DEVELOPMENT CHEAT SHEET — CS 4032 | MongoDB · Express · React · Angular

MONGODB

Core Concepts
• NoSQL Document Database (JSON-like BSON)
• Server → Databases → Collections → Documents
DB: Container of collections
Collection: Group of documents (≈ Table)
Document: JSON object (≈ Row); has _id field
Field: Key-value pair (≈ Column)

BSON Data Types
String, Number, Boolean, Date, Array, Object, ObjectId, Null, Regex, Binary

Shell Commands
```
show dbs
use <dbName>
show collections
db.dropDatabase()
```

CRUD Operations
```
db.col.insertOne({name:'Ali', age:20})
db.col.insertMany([{...},{...}])
db.col.find({status:'A'})
db.col.find({},{name:1, id:0})  // projection
db.col.findOne({_id: ObjectId('...')})
db.col.updateOne({name:'Ali'},{$set:{age:21}})
db.col.updateMany({status:'A'},{$set:{x:1}})
db.col.deleteOne({name:'Ali'})
db.col.deleteMany({age:{$lt:18}})
db.col.drop()
```

Query Operators
$eq / $ne: equal / not equal
$gt / $gte / $lt / $lte: comparisons
$in / $nin: value in/not-in array
$and / $or / $not / $nor: logical ops
$exists: field exists check
```
db.col.find({price:{$gte:10,$lte:50}})
db.col.find({$or:[{a:1},{b:2}]})
```

Update Operators
$set: set field value
$unset: remove field
$inc: increment number
$push: add to array
$pull: remove from array
$currentDate: set to current date

Embedded Documents & Arrays
```
db.col.find({'address.city':'Lahore'})
db.col.find({tags:{$all:['NoSQL']}})
db.col.find({comments:{$elemMatch:{
  date:'04-12-2020'}}})
```

Aggregation / JOIN ($lookup)
```
db.articles.aggregate([
  {$lookup:{from:'User',
   localField:'Author',
   foreignField:'_id',as:'user'}},
  {$unwind:'$user'},
  {$project:{title:1,
   Author:'$user.Name'}}
])
```

Indexing
```
db.users.createIndex({lastName:1})
db.users.createIndex({email:1},
  {unique:true, name:'EmailIdx'})
```

MongoDB + Node.js (Mongoose)
```
const mongoose = require('mongoose')
mongoose.connect('mongodb://localhost/db')
const Schema = new mongoose.Schema({
  name:String, email:{type:String,
  required:true,unique:true}})
module.exports = mongoose.model('User',Schema)
```

Relational vs MongoDB
Table: Collection | Row → Document
Column: Field | JOIN → $lookup
GROUP BY: Aggregation Pipeline

EXPRESS.JS (Part-1)

Basic Route
```
app.get('/path', (req,res) => {
  res.json({msg:'Hello'})
})
app.post('/path', (req,res) => {...})
app.put('/user', (req,res) => {...})
app.delete('/user', (req,res) => {...})
```

Request Object
req.params: path params (/user/:id)
req.query: query string (?name=Ali)
req.body: request body (POST/PUT)
req.get(h): get header value
```
app.get('/users/:id/books/:bid',
  (req,res)=>res.send(req.params))
```

Response Methods
res.send(): send HTML/text
res.json(obj): send JSON (sets content-type)
res.status(n): set HTTP status code
res.type('text'): set content-type
res.redirect(url): redirect
res.sendStatus(n): status + default text
```
res.status(500).json({error:'msg'})
```

Middleware
Function with (req, res, next)
```
app.use(express.json())          //parse JSON
app.use(express.urlencoded({extended:true}))
app.use(express.static('public'))
```

REACT

Core Concepts
• JS library for building UI (View in MVC)
• Component-based, Virtual DOM, JSX
• Re-renders only changed parts

JSX
JavaScript XML — HTML-like syntax in JS
```
const el = <h1>Hello {name}</h1>
const list = (
  <ul><li>Apple</li><li>Banana</li></ul>)
// Self-closing: <input type='text' />
```

Functional Component
```
function Welcome(props) {
  return <h1>Hello {props.name}</h1>
}
// Usage: <Welcome name='Ali' />
```

Class Component
```
class Welcome extends React.Component {
  render() {
    return <h1>{this.props.name}</h1>
  }
}
```

Props (Parent → Child, immutable)
```
function App() {
  return <Greeting name='Ali' age={25}/>
}
function Greeting({name,age}) {
  return <p>{name} is {age}</p>
}
```

State (Class)
```
class Counter extends React.Component {
  constructor(props) {
    super(props)
    this.state = {count:0}
  }
  render() {
    return <button onClick={()=>
      this.setState({count:this.state.count+1})}
    >{this.state.count}</button>
  }
}
```

useState Hook (Functional)
```
import {useState} from 'react'
function Counter() {
  const [count, setCount] = useState(0)
  return <button onClick={()=>setCount(count+1)}>
    {count}</button>
}
```

Lifecycle (Class)
componentDidMount: after first render (API calls)
componentDidUpdate: after re-render
componentWillUnmount: cleanup before removal
```
componentDidMount() {
  fetch('/api/data').then(r=>r.json())
    .then(d=>this.setState({data:d}))
}
```

useEffect Hook
```
import {useEffect} from 'react'
useEffect(() => {
  fetch('/api').then(...)
}, [])  // [] = run once on mount
```

Events
```
// camelCase, {}, arrow fn
<button onClick={() => alert('hi')}>
<input onChange={e=>setVal(e.target.value)}/>
// Pass args:
<button onClick={()=>shoot('Goal')}>
```

Controlled Forms
```
const [name,setName] = useState('')
<input value={name}
  onChange={e=>setName(e.target.value)}/>
<button onClick={handleSubmit}>Send</button>
```

Uncontrolled Forms (ref)
```
const inputRef = React.createRef()
<input type='text' ref={inputRef}/>
// Get value: inputRef.current.value
```

Conditional Rendering
```
// Ternary
{isLoggedIn ? <Logout/> : <Login/>}
// && operator
{count > 0 && <p>You have {count} msgs</p>}
// Return null to hide
if(!show) return null
```

Lists & Keys
```
const items = ['Apple','Banana','Cherry']
<ul>
  {items.map((item,i) =>
    <li key={i}>{item}</li>)}
</ul>
// Keys must be unique siblings
```

React Fragments
```
// Avoid extra <div> wrapper
return (
  <React.Fragment>
    <h1>Title</h1>
    <p>Content</p>
  </React.Fragment>)
// Short: <> ... </>
```

React Router
```
npm install react-router-dom
import {BrowserRouter as Router,
```

ANGULAR

TypeScript Basics
```
let x: number = 5
let s: string = 'hello'
let b: boolean = true
let arr: number[] = [1,2,3]
function add(a:number, b:number): number {
  return a+b }
// Optional: b?: number
// Access: public / private / protected
```

@NgModule (app.module.ts)
```
@NgModule({
  declarations: [AppComponent,
    MyComponent],    // components/pipes
  imports: [BrowserModule,
    HttpClientModule,
    RouterModule.forRoot(routes)],
  providers: [MyService],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

@Component Decorator
```
@Component({
  selector: 'app-my',
  templateUrl: './my.component.html',
  styleUrls: ['./my.component.css']
})
export class MyComponent implements OnInit {
  title = 'Hello'
  ngOnInit(): void { /* init logic */ }
}
```

Template Syntax
{{ expr }}: Interpolation
[prop]='expr': Property binding
(event)='fn()': Event binding
[(ngModel)]='prop': Two-way binding
#varName: Template reference variable
```
<input [(ngModel)]='name'>
<p>Hello {{name}}</p>
<button (click)='save()'>Save</button>
<img [src]='imageUrl'>
```

Structural Directives
```
<!-- *ngFor -->
<li *ngFor='let item of items'>
  {{item.name}}</li>
<li *ngFor='let i of items;
  let idx=index'>{{idx}}.{{i}}</li>

<!-- *ngIf -->
<p *ngIf='items.length==0'>Empty</p>
<p *ngIf='x; else noX'>Has X</p>
<ng-template #noX><p>No X</p></ng-template>

<!-- ngSwitch -->
<div [ngSwitch]='day'>
  <span *ngSwitchCase='1'>Mon</span>
  <span *ngSwitchDefault>Other</span>
</div>
```

Attribute Directives
```
<p [ngStyle]="{color:'red'}">Text</p>
<p [style.color]="'blue'">Text</p>
<p [ngClass]="{active:isActive,
  disabled:!isActive}">Text</p>
```

Pipes
```
{{entry.date | date:'M/d/yyyy'}}
{{3.14159 | number:'1.2-2'}}
{{price | currency:'USD'}}
{{'hello' | uppercase}}
{{items | json}}
```
date: format dates
number: decimal places
currency: money format
uppercase/lowercase: case transform
json: serialize object

@Input() — Parent → Child
```
// child.component.ts
import {Input} from '@angular/core'
@Input() title: string = ''
@Input() items: Item[] = []

// parent template
<app-child [title]='myTitle'
  [items]='myItems'>
</app-child>
```

@Output() — Child → Parent
```
// child.component.ts
import {Output,EventEmitter} from '@angular/core'
@Output() itemAdded =
  new EventEmitter<Item>()

addItem() {
  this.itemAdded.emit(newItem)
}

// parent template
<app-child (itemAdded)='onAdd($event)'>
</app-child>

// parent component
onAdd(item: Item) {
  this.items.push(item)
}
```

@Injectable Service
```
@Injectable({ providedIn: 'root' })
export class DataService {
  constructor(private http: HttpClient){}
  getData(): Observable<Item[]> {
    return this.http.get<Item[]>('/api/items')
  }
  postData(item:Item): Observable<Item> {
```

```
app.use((req,res,next) => {
  console.log(req.method,req.url)
  next()  // MUST call next!
})
```

Express Router (Modular)

```
// users.js
const router = express.Router()
router.get('/', (req,res) => {...})
router.post('/', (req,res) => {...})
module.exports = router
// app.js
const users = require('./users')
app.use('/users', users)
```

Error Handling Middleware

```
app.use((err,req,res,next) => {
  res.status(500).json({error:err.msg})
})
// call next(err) to trigger it
```

JWT Auth

```
const jwt = require('jsonwebtoken')
const token = jwt.sign({user},secret)
const payload = jwt.verify(token,secret)
// Auth header: Bearer <token>
```

1. The Backend: server.js (Express + MongoDB)

```
// server.js
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');

const app = express();
const PORT = 5000;

// Middleware
app.use(cors()); // Allows React to communicate with Express
app.use(express.json()); // Parses incoming JSON data from
the client

// 1. Create MongoDB Connection
mongoose.connect('mongodb://localhost:27017/my_app_db')
  .then(() => console.log('Connected to MongoDB'))
  .catch((err) => console.error('MongoDB connection error:',
err));

// Define a Mongoose Schema and Model
const userSchema = new mongoose.Schema({
  name: String,
  email: String
});
const User = mongoose.model('User', userSchema);

// 2. GET Route (Fetch data from API)
app.get('/api/users', async (req, res) => {
  try {
    const users = await User.find(); // Get all users from
MongoDB
    res.json(users);                 // Send them back to
React
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch data' });
  }
});

// 3. POST Route (Get data from client side & insert in
Mongo)
app.post('/api/users', async (req, res) => {
  try {
    // req.body contains the data sent from the React form
    const newUser = new User({
      name: req.body.name,
      email: req.body.email
    });

    await newUser.save(); // Insert into MongoDB
    res.status(201).json(newUser); // Send back the created
user
  } catch (err) {
    res.status(500).json({ error: 'Failed to save data' });
  }
});

// Start Server
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

2. The Frontend: App.js (React)

```
// App.js
import React, { useState, useEffect } from 'react';

function App() {
  // State for the form inputs
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');

  // State to hold the fetched data
  const [users, setUsers] = useState([]);

  // 1. Fetch data from API when component loads
  useEffect(() => {
    fetchUsers();
  }, []);

  const fetchUsers = async () => {
    try {
      const response = await
fetch('http://localhost:5000/api/users');
      const data = await response.json();
      setUsers(data);
    } catch (error) {
      console.error('Error fetching data:', error);
    }
  };
```

```
 Routes, Route, Link} from 'react-router-dom'

<Router>
  <Link to='/about'>About</Link>
  <Routes>
    <Route path='/' element={<Home/>}/>
    <Route path='/about' element={<About/>}/>
    <Route path='*' element={<NotFound/>}/>
  </Routes>
</Router>
```

Navigate in Code

```
import {useNavigate} from 'react-router-dom'
const navigate = useNavigate()
navigate('/dashboard')
```

Route Params

```
<Route path='/user/:id' element={<User/>}/>
import {useParams} from 'react-router-dom'
const {id} = useParams()
```

Fetch API

```
fetch('http://api.example.com/users')
  .then(res => res.json())
  .then(data => setState({data}))
  .catch(console.log)
```

Axios

```
npm install axios
axios.get('/api/users').then(res=>{
  setState({data:res.data})})
axios.post('/api',{name,msg})
  .then(()=>navigate('/'))
```

Context API (State Mgmt)

```
const ThemeCtx = React.createContext('light')
// Provider (parent):
<ThemeCtx.Provider value='dark'>
  <App/>
</ThemeCtx.Provider>
// Consumer (child):
const theme = useContext(ThemeCtx)
```

CSS Styling

```
// Inline
const style = {color:'red',fontSize:16}
<h1 style={style}>Hello</h1>
// CSS file: import './App.css'
// Bootstrap: import in index.html
```

Component Communication

- Parent→Child: props
- Child→Parent: callback fn via props
- Siblings: lift state to parent
- Any: Context API or Redux

EXPRESS.JS (Part-2)

```
// …continued [2. The Frontend: App.js (React)]
  // 2. Create a form in React & Send Data to Express
  const handleSubmit = async (e) => {
    e.preventDefault(); // Prevent page reload

    const formData = { name, email };

    try {
      const response = await
fetch('http://localhost:5000/api/users', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(formData) // Send data to
client side (Express)
      });

      const newUser = await response.json();

      // Update UI with the new user without reloading
      setUsers([...users, newUser]);

      // Clear form fields
      setName('');
      setEmail('');
    } catch (error) {
      console.error('Error submitting form:', error);
    }
  };

  return (
    <div style={{ padding: '20px', fontFamily: 'Arial' }}>
      <h2>Add a New User</h2>

      {/* The React Form */}
      <form onSubmit={handleSubmit} style={{ marginBottom:
'20px' }}>
        <div>
          <label>Name: </label>
          <input
            type="text"
            value={name}
            onChange={(e) => setName(e.target.value)}
            required
          />
        </div>
        <div style={{ marginTop: '10px' }}>
          <label>Email: </label>
          <input
            type="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            required
          />
        </div>
        <button type="submit" style={{ marginTop: '10px'
}}>Submit</button>
      </form>

      <hr />

      {/* Display the Fetched Data */}
```

```
    return this.http.post<Item>('/api',item)
  }
}

// Inject in component
constructor(private ds: DataService){}
ngOnInit(){
  this.ds.getData().subscribe(
    data => this.items = data)
}
```

HttpClient (import HttpClientModule)

```
// GET
this.http.get<T>('/api/items')
// POST
this.http.post<T>('/api',body)
// PUT
this.http.put<T>('/api/id',body)
// DELETE
this.http.delete('/api/id')
```

Angular Routing

```
// app-routing.module.ts
const routes: Routes = [
  {path:'',component:HomeComponent},
  {path:'about',component:AboutComponent},
  {path:'user/:id',component:UserComponent},
  {path:'**',component:NotFoundComponent}
]

// template navigation
<a routerLink='/about'>About</a>
<a [routerLink]="['/user',userId]">

// read route params
constructor(private route: ActivatedRoute){}
ngOnInit(){
  const id = this.route.snapshot
    .paramMap.get('id')
}

// navigate in code
constructor(private router: Router){}
this.router.navigate(['/home'])
```

router-outlet & routerLink

```
// app.component.html
<nav>
  <a routerLink='/'>Home</a>
  <a routerLink='/about'>About</a>
</nav>
<router-outlet></router-outlet>
```

Dependency Injection

providers:[]: in @NgModule or @Component
providedIn:'root': singleton app-wide
constructor(private s:Service){}: inject

```
// Provider with value token
providers:[{provide:'API_URL',
  useValue:'http://localhost:3000'}]
// Inject with @Inject
constructor(@Inject('API_URL') url:string)
```

Key Decorators Summary

@Component: defines a component (selector,template,style)
@NgModule: defines a module (declarations,imports,exports)
@Injectable: marks class for DI
@Input(): receive data from parent
@Output(): emit events to parent
@Inject(): inject non-class tokens

Lifecycle Hooks

ngOnInit: after component init (use for API calls)
ngOnChanges: when @Input values change
ngOnDestroy: cleanup before removal
ngAfterViewInit: after view initialized

Dev Proxy Setup

```
// proxy.conf.json
{"/api":{"target":
  "http://localhost:3000",
  "secure":false}}
// package.json scripts:
"start":"ng serve
  --proxy-config proxy.conf.json"
```

Environment Variables

```
// .env file
DBURL=mongodb://localhost/mydb
JWT_SECRET=mysecret
// In Node.js
process.env.DBURL
require('dotenv').config()
```

EXPRESS.JS (Part-3)

```
// …continued [2. The Frontend: App.js (React)]
  //
<h2>Database Records</h2>
      <ul>
        {users.map((user) => (
          <li key={user._id}>
            <strong>{user.name}</strong> - {user.email}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```